PulseStream — A Guided Build (Tutorial)

A companion to the High-Level Design. This walks you from an empty folder to a running vertical slice (live events → DuckDB → API), then hands you a guided path for the rest.

How to use this tutorial

This is a **guided build, not a finished repo**. It gives you:

- the exact project scaffold and tooling,
- the patterns that are genuinely easy to get wrong (async orchestration, SSE, DuckDB writes, FastAPI lifespan), written out, and
- a working test harness so you can refactor with confidence.

It deliberately leaves the *interesting* parts for you. Watch for these markers:

- `# TODO:` — code you write. Hints are in the comments.
- **Your turn** — a slightly larger exercise, usually a real engineering decision.
- **Definition of done** — how you know a part is finished before moving on.

Prerequisites: Python 3.12+, Docker, git, a GitHub account. Comfortable with the command line and basic async. **Running example source:** Wikimedia EventStreams (free, no auth, robust). A crypto WebSocket swaps in behind the same interface later — that's the whole point of Part 1.1.

Scope: Parts 0–2 (Phase 0 + the Phase 1 slice + tests) are covered in depth. Part 3 is a lighter guided path for Phases 2–5; lean on the HLD for the design.

Part 0 — Foundations (Phase 0)

Goal: a packaged project with linting, typing, tests, and a **green CI on an empty app**. Get the machinery working *before* any logic — it's far cheaper to debug CI now than later.

0.1 Initialize with uv

```bash
uv init --package pulsestream
cd pulsestream

# runtime deps
uv add fastapi uvicorn duckdb pydantic pydantic-settings httpx httpx-sse \
       structlog prometheus-client

# dev deps
uv add --dev pytest pytest-asyncio pytest-cov ruff mypy pre-commit pandera
```

`uv add` writes to `pyproject.toml` and pins everything in `uv.lock`. Commit both.

0.2 Lay out the package

Create the structure (empty `__init__.py` in each `pulsestream/*` dir):

```
pulsestream/
  pyproject.toml
  uv.lock
  src/pulsestream/
    __init__.py
    config.py
    models.py
    pipeline.py
    sources/      __init__.py  base.py  wikimedia.py  fake.py
    storage/      __init__.py  duck.py
    api/          __init__.py  app.py
  scripts/        capture_sample.py
  tests/
    __init__.py
    fixtures/
    test_smoke.py
  .github/workflows/ci.yml
```

0.3 Configure the tooling

Append to `pyproject.toml`:

```toml
```

```
[tool.ruff]
line-length = 100

[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B"]   # errors, pyflakes, import-sort, pyupgrad

[tool.mypy]
python_version = "3.12"
strict = true

[tool.pytest.ini_options]
asyncio_mode = "auto"                 # lets `async def test_*` run without a d
addopts = "--cov=pulsestream --cov-report=term-missing"
```

A `.pre-commit-config.yaml` so problems are caught before they hit CI:

yaml

```yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.8.0
    hooks:
      - id: ruff
        args: [--fix]
      - id: ruff-format
```

Then `uv run pre-commit install`.

0.4 Prove the harness

`tests/test_smoke.py`:

python

```python
def test_smoke():
    assert 1 + 1 == 2
```

```bash
uv run ruff check .
uv run mypy src
uv run pytest
```

All three should pass on the empty project.

0.5 Green CI on an empty app

`.github/workflows/ci.yml`:

```yaml
```

```yaml
name: ci
on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.12", "3.13"]
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v5
        with:
          python-version: ${{ matrix.python-version }}
          enable-cache: true
      - run: uv sync --dev
      - run: uv run ruff check .
      - run: uv run ruff format --check .
      - run: uv run mypy src
      - run: uv run pytest
```

Push to GitHub. In **Settings → Branches**, add a rule requiring this check to pass before merging to `main`.

> **Definition of done:** a green CI badge on a repo that does nothing yet, and `pre-commit` running locally on every commit.

Part 1 — The vertical slice: live events → DuckDB → API (Phase 1)

Goal: real events from the wild land in DuckDB and a `/stats/latest` endpoint returns honest numbers. One process, in-process queue, no broker yet.

1.1 Abstract the source

Everything downstream should be ignorant of *where* events come from. That's a `Protocol` (structural typing — no inheritance needed).

`src/pulsestream/sources/base.py`:

python

```python
from collections.abc import AsyncIterator
from typing import Protocol

from pulsestream.models import RawEvent


class EventSource(Protocol):
    def stream(self) -> AsyncIterator[RawEvent]:
        """Yield raw events forever (until cancelled)."""
        ...
```

> **Why:** when Part 1 works on Wikimedia and you later want crypto, you write a new class with a `stream()` method and change *one line* of wiring. No `if source == "crypto"` branching anywhere.

1.2 Model your data

`src/pulsestream/models.py`:

python

```python
import json
from datetime import datetime
from uuid import UUID, uuid4

from pydantic import BaseModel, Field


class RawEvent(BaseModel):
    """Source-agnostic envelope. Never changes between sources."""
    ingest_id: UUID = Field(default_factory=uuid4)
    source: str
    received_at: datetime
    schema_version: int
    raw_payload: str            # keep the original; parse later


class WikiEvent(BaseModel):
    """Validated, source-specific domain event."""
    event_id: str
    entity: str                 # e.g. f"{wiki}:{title}"
    event_ts: datetime
    byte_delta: int
    is_bot: bool
    received_at: datetime

    @classmethod
    def from_raw(cls, raw: RawEvent) -> "WikiEvent | None":
        data = json.loads(raw.raw_payload)
        # The recentchange schema gives you, among others:
        #   data["type"]      -> "edit" | "new" | "log" | "categorize" ...
        #   data["title"], data["wiki"], data["bot"]
        #   data["timestamp"] -> unix seconds (int)
        #   data["length"]    -> {"old": int, "new": int}  (absent for some typ
```

```
        #
        # TODO:
        #   1. Decide which event types you keep (return None to drop the rest)
        #   2. Compute byte_delta from length.new - length.old (guard missing k
        #   3. Build `entity`, parse event_ts from the unix timestamp (UTC!).
        raise NotImplementedError
```

> **Your turn:** implement `from_raw`. Returning `None` for events you don't care about is your first filtering decision — make it deliberate.

1.3 Implement the Wikimedia source

The SSE read loop is the part people fumble. Here it is, working:

`src/pulsestream/sources/wikimedia.py`:

python

```python
from collections.abc import AsyncIterator
from datetime import UTC, datetime

import httpx
from httpx_sse import aconnect_sse

from pulsestream.models import RawEvent

STREAM_URL = "https://stream.wikimedia.org/v2/stream/recentchange"


class WikimediaSource:
    async def stream(self) -> AsyncIterator[RawEvent]:
        async with httpx.AsyncClient(timeout=None) as client:
            async with aconnect_sse(client, "GET", STREAM_URL) as events:
                async for sse in events.aiter_sse():
                    if sse.event == "message" and sse.data:
                        yield RawEvent(
                            source="wikimedia",
                            received_at=datetime.now(UTC),
                            schema_version=1,
                            raw_payload=sse.data,
                        )
```

> **Your turn (reliability):** as written, a dropped connection ends the stream. Wrap the two `async with` blocks in `while True:` with `try/except (httpx.HTTPError, httpx.StreamError)`, and back off before reconnecting — exponential with jitter, capped (e.g. `min(cap, base * 2**attempt) + random`). Reset the attempt counter after a successful run of events. This is exactly the kind of "engineering, not data" work the project is for.

`src/pulsestream/pipeline.py`:

```python
```

```python
import asyncio

import duckdb

from pulsestream.models import RawEvent, WikiEvent
from pulsestream.sources.base import EventSource
from pulsestream.storage.duck import write_batch


async def run_producer(source: EventSource, queue: asyncio.Queue[RawEvent | Non
    async for raw in source.stream():
        await queue.put(raw)        # blocks when the queue is full == backpres


async def run_consumer(
    queue: asyncio.Queue[RawEvent | None],
    con: duckdb.DuckDBPyConnection,
    batch_size: int = 50,
    flush_interval: float = 2.0,
) -> None:
    batch: list[WikiEvent] = []
    while True:
        raw = await queue.get()
        if raw is None:             # sentinel -> flush & stop (used by tests/s
            if batch:
                write_batch(con, batch)
            queue.task_done()
            return
        event = WikiEvent.from_raw(raw)
        if event is not None:
            batch.append(event)
        if len(batch) >= batch_size:
            write_batch(con, batch)
```

```
            batch.clear()
        queue.task_done()
```

> **Your turn (micro-batching):** the consumer above only flushes when the batch fills — a quiet stream could sit unwritten for a long time. Add a time-based flush every `flush_interval` seconds. Hint: replace the bare `await queue.get()` with `asyncio.wait_for(queue.get(), timeout=...)` and flush on `TimeoutError`, *or* run a small separate flusher task. Either is a fine answer — know why you picked yours.

The `maxsize` you give the queue is your backpressure knob (Part 1.7). A bounded queue + a blocking `put` means a slow consumer naturally slows the producer instead of blowing up memory.

1.5 Persist to DuckDB

`src/pulsestream/storage/duck.py`:

```python
```

```python
import duckdb

from pulsestream.models import WikiEvent


def init_db(con: duckdb.DuckDBPyConnection) -> None:
    con.execute(
        """
        CREATE TABLE IF NOT EXISTS events (
            event_id    VARCHAR,
            entity      VARCHAR,
            event_ts    TIMESTAMPTZ,
            byte_delta  INTEGER,
            is_bot      BOOLEAN,
            received_at TIMESTAMPTZ
        )
        """
    )


def write_batch(con: duckdb.DuckDBPyConnection, batch: list[WikiEvent]) -> None
    con.executemany(
        "INSERT INTO events VALUES (?, ?, ?, ?, ?, ?)",
        [
            (e.event_id, e.entity, e.event_ts, e.byte_delta, e.is_bot, e.receiv
            for e in batch
        ],
    )
```

> **Gotcha — don't block the event loop.** DuckDB calls are synchronous. In Part 1.6 they run inside an async app; a slow query would stall ingestion *and* the API. For the slice it's fine (queries are tiny), but the correct fix is to wrap DuckDB work in `await`

> `asyncio.to_thread(...)`. Note it now; apply it when a query gets heavy. (Add it to your `decisions/` ADRs.)
> **Gotcha — one writer.** DuckDB allows a single read-write process. When you split the API into its own process later, open it `read_only=True`.

1.6 Serve it (FastAPI lifespan starts the pipeline)

`src/pulsestream/api/app.py`:

```python
```

```python
import asyncio
from contextlib import asynccontextmanager

import duckdb
from fastapi import FastAPI, Request

from pulsestream.config import settings
from pulsestream.models import RawEvent
from pulsestream.pipeline import run_consumer, run_producer
from pulsestream.sources.wikimedia import WikimediaSource
from pulsestream.storage.duck import init_db


@asynccontextmanager
async def lifespan(app: FastAPI):
    con = duckdb.connect(settings.db_path)
    init_db(con)
    app.state.con = con

    queue: asyncio.Queue[RawEvent | None] = asyncio.Queue(maxsize=settings.queu
    source = WikimediaSource()
    producer = asyncio.create_task(run_producer(source, queue))
    consumer = asyncio.create_task(run_consumer(queue, con))

    yield  # ---- app runs ----

    producer.cancel()
    consumer.cancel()
    await asyncio.gather(producer, consumer, return_exceptions=True)
    con.close()


app = FastAPI(lifespan=lifespan)
```

```python
@app.get("/health")
async def health() -> dict[str, str]:
    return {"status": "ok"}


@app.get("/stats/latest")
async def stats_latest(request: Request) -> dict[str, object]:
    con: duckdb.DuckDBPyConnection = request.app.state.con
    count, latest = con.execute(
        "SELECT count(*), max(event_ts) "
        "FROM events WHERE event_ts > now() - INTERVAL 1 MINUTE"
    ).fetchone()
    return {"events_last_min": count, "latest_event_ts": latest}
```

`src/pulsestream/config.py`:

```python
from pydantic_settings import BaseSettings


class Settings(BaseSettings):
    db_path: str = "pulsestream.duckdb"
    queue_maxsize: int = 1000

    class Config:
        env_prefix = "PULSE_"          # PULSE_DB_PATH, PULSE_QUEUE_MAXSIZE


settings = Settings()
```

> **Your turn:** add a `/timeseries?bucket=10s&minutes=10` endpoint that returns per-bucket counts. DuckDB's `time_bucket(INTERVAL '10 seconds', event_ts)` makes this a one-line `GROUP BY`. Return a Pydantic model so your OpenAPI docs stay accurate.

1.7 Run it end to end

```bash
uv run uvicorn pulsestream.api.app:app --reload
```

Then, after ~10 seconds of ingestion:

```bash
curl localhost:8000/health
curl localhost:8000/stats/latest
```

Watch `events_last_min` climb. If `from_raw` still raises `NotImplementedError`, the consumer task will error — check the logs; that's your cue to finish 1.2.

> **Definition of done:** live Wikipedia edits are landing in DuckDB and `/stats/latest` returns a non-zero, increasing count with a fresh timestamp.

Part 2 — Make it trustworthy (replay + first tests)

A live source can't be in your tests. The fix is **replay**: record a sample once, test against it forever.

2.1 Capture a sample

`scripts/capture_sample.py`:

```python
import asyncio
from pathlib import Path

from pulsestream.sources.wikimedia import WikimediaSource


async def main(n: int = 500, path: str = "tests/fixtures/sample_events.jsonl"):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    src = WikimediaSource()
    count = 0
    with open(path, "w") as f:
        async for raw in src.stream():
            f.write(raw.model_dump_json() + "\n")
            count += 1
            if count >= n:
                break
    print(f"wrote {count} events to {path}")


if __name__ == "__main__":
    asyncio.run(main())
```

```bash
uv run python scripts/capture_sample.py
```

Commit the fixture — it makes CI deterministic and offline.

2.2 Build a FakeSource

`src/pulsestream/sources/fake.py`:

```python
from collections.abc import AsyncIterator
from pathlib import Path

from pulsestream.models import RawEvent


class FakeSource:
    def __init__(self, events: list[RawEvent]) -> None:
        self._events = events

    async def stream(self) -> AsyncIterator[RawEvent]:
        for raw in self._events:
            yield raw


def load_fixture(path: str) -> list[RawEvent]:
    return [RawEvent.model_validate_json(line) for line in Path(path).read_text
```

Because it satisfies the same `EventSource` shape, it drops into the real pipeline with zero changes.

2.3 Write the integration test

`tests/test_pipeline.py`:

```python
```

```python
import asyncio

import duckdb

from pulsestream.pipeline import run_consumer, run_producer
from pulsestream.sources.fake import FakeSource, load_fixture
from pulsestream.storage.duck import init_db


async def test_pipeline_persists_valid_events(tmp_path):
    con = duckdb.connect(str(tmp_path / "test.duckdb"))
    init_db(con)

    raws = load_fixture("tests/fixtures/sample_events.jsonl")
    queue: asyncio.Queue = asyncio.Queue()

    await run_producer(FakeSource(raws), queue)  # finite source -> completes
    await queue.put(None)                        # sentinel stops the consumer
    await run_consumer(queue, con)

    rows = con.execute("SELECT count(*) FROM events").fetchone()[0]
    assert rows > 0
    # TODO: tighten this. Count how many fixture events your `from_raw` should
    # keep, and assert equality. Add a unit test for `from_raw` covering: an
    # accepted edit, a dropped type, and an event with a missing `length` key.
```

> **Definition of done:** `uv run pytest` is green against the fixture, with at least one unit test pinning down `from_raw`'s filtering and `byte_delta` math.

Part 3 — The guided path for Phases 2–5

You now have the spine. Each phase below is one demoable increment. I give the goal, the one new idea that matters, the gotcha, and the finish line — the design rationale is in the HLD.

Phase 2 — Storage + analytics

- **Goal:** durable raw events + fast charts.
- **Key idea:** append events to **partitioned Parquet** (`raw/dt=YYYY-MM-DD/`) as your immutable source of truth, and let DuckDB query the files directly with `read_parquet('raw/**/*.parquet')`. Build derived tables (`agg_minutely`) from there. Add a Streamlit dashboard that reads **through your API**, and a scheduled compaction job (APScheduler) to merge small files.
- **Gotcha:** writing one Parquet file per event destroys read performance. Write in micro-batches (you already batch in the consumer — reuse it). Use a columnar writer (Polars `write_parquet` or pyarrow).
- **Done:** a chart of edits/min over the last hour, served from the API, backed by aggregates DuckDB computed over Parquet.

Phase 3 — Anomaly detection v1

- **Goal:** flag unusual activity in near-real-time.
- **Key idea:** maintain rolling features per entity (rate, EWMA, rolling std), then flag points outside `EWMA ± k·std` (start with k=3). Pure functions — easy to unit test. Write flags to an `anomalies` table and expose `/anomalies`.
- **Gotcha:** you have **no labels**. Evaluate by injecting *synthetic* spikes into a replayed stream and measuring recall/precision — exactly what Figure 3 in the HLD illustrates (and note the failure mode: a spike can hide inside a band that your own earlier outliers widened).
- **Done:** a synthetic-anomaly test that asserts your detector catches injected spikes, plus an anomalies panel on the dashboard.

Phase 4 — CI/CD + observability (productionize)

- **Goal:** ship it, and be able to see it run.
- **Key ideas:** multi-stage `Dockerfile` (small, non-root) per service; `docker-compose.yml` to bring everything up with one command; extend CI to build + push images to GHCR and **deploy on a `v*` tag**; expose Prometheus metrics (`prometheus-client`) — `processing_lag_seconds`, `ingest_events_total`, `anomalies_detected_total`, `errors_total`.
- **Gotcha:** keep CI fast — cache the uv env and Docker layers, and never run the live source in CI (you already have the fixture).
- **Done:** a public URL from a `git tag`, a `/metrics` endpoint, and a Grafana (or Streamlit) view of lag and event rate.

Phase 5 — Streaming decoupling (the "real" milestone)

- **Goal:** survive an ingestion crash without losing the processing pipeline, and be able to replay.
- **Key idea:** put a broker between ingestion and processing — **Redpanda** (Kafka-compatible single binary) or **Redis Streams**. Ingestion becomes a producer; processing becomes a consumer group reading offsets. This is where your `asyncio.Queue` boundary pays off: you swap the queue for the broker and split the one process into two services.
- **Gotcha:** decide your delivery semantics explicitly. Target **at-least-once + idempotent writes** (dedupe on `event_id`) rather than chasing exactly-once.
- **Done:** ingestion and processing run as separate containers; killing ingestion and restarting it does not stop processing, and you can replay a past window from the broker.

Appendix — gotchas & tips

- **Async pitfalls.** A synchronous call inside an `async def` blocks the *whole* event loop. Anything CPU-bound or blocking (DuckDB, file I/O, model inference) belongs in `asyncio.to_thread(...)`. Always `await` your tasks (or cancel + gather them) on shutdown, or you'll leak warnings and half-written batches.
- **DuckDB concurrency.** One read-write process; others open `read_only=True`. Within a process, don't share one connection across threads casually — give the threadpool path its own cursor or serialize writes through the consumer.
- **Timestamps.** Store `event_ts` (when it happened) and `received_at` (when you got it) separately, both UTC and timezone-aware. You need the gap to compute lag.
- **Keep CI under ~2 minutes.** Cache uv + Docker layers, keep the fixture small, and never hit the network in tests.
- **Debugging SSE.** `curl -N https://stream.wikimedia.org/v2/stream/recentchange` shows you raw events in the terminal — handy for sanity-checking your parser.
- **When you're stuck.** Each part has a *Definition of done*; if reality and that diverge, the bug is between your last green checkpoint and now. And if you'd rather see a fuller worked version of any single step here, ask for that step — this tutorial is intentionally scaffolded, not exhaustive.

```
Start at Part 0 and don't write a line of pipeline logic until CI is green on
the
empty app. That one habit will save you more time than anything else here.
```